

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING & TECHNOLOGY



(SUBJECT : ADVANCED MACHINE LEARNING)
(SUBJECT CODE : 24SAL051)

A REPORT ON

**“Comparative Analysis of ML Algorithms on the
Wine Dataset”**

Submitted in the partial fulfilment of the requirements for the fifth semester

BACHELOR OF TECHNOLOGY IN AIML

Submitted by

THRISHIKA ANCHAN

01SU24AI110

UNDER THE GUIDENCE OF

Prof . Mahesh Kumar V B

Department of AIML

**SRINIVAS UNIVERSITY INSTITUTE OF ENGINEERING &
TECHNOLOGY MUKKA, MANGALURU-574146**

2026

SRINIVAS UNIVERSITY
INSTITUTE OF ENGINEERING AND TECHNOLOGY
MUKKA, MANGALURU-574146

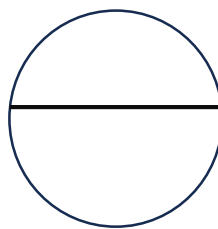


Department of Artificial Intelligence and Machine Learning

CERTIFICATE

This is to certify that, Ms. Thrishika Anchan (**01SU24AI110**) has satisfactorily completed the assessment (Report) in **Advanced Machine Learning (24SAL051)** prescribed by the Srinivas University for the 5th semester B. Tech course during the year 2026.

MARKS AWARDED



Staff In charge

Name: Prof . Mahesh Kumar V B

Assistant Professor, Department of AIML

Date: 11.09.2026

1. Abstract

Machine Learning (ML) has become an important technology for analyzing large amounts of data, identifying patterns, and making reliable predictions. The performance of a machine learning system depends significantly on the choice of an appropriate algorithm. This project focuses on the comparative analysis of different machine learning classification algorithms using the Wine Dataset. The Wine Dataset consists of multiple chemical and physical characteristics of wine samples, which can be used to classify the samples into different wine categories.

In this study, three supervised machine learning algorithms, namely Decision Tree, Gaussian Naive Bayes, and K-Nearest Neighbors (KNN), are applied to the Wine Dataset. The dataset is first examined and preprocessed to make it suitable for machine learning. The data is then divided into training and testing sets so that the models can be trained and evaluated effectively. Feature scaling is also performed where required to ensure that features with different numerical ranges do not negatively affect the performance of the algorithms.

2. Introduction

Machine Learning is a branch of Artificial Intelligence that enables computers to learn from data and make predictions or decisions without being explicitly programmed. It is widely used for classification, prediction, and data analysis in various fields.

In this project, the **Wine Dataset** is selected to study and compare different machine learning classification algorithms. The dataset contains various chemical characteristics of wine samples that help in classifying them into different categories. Three algorithms—**Decision Tree, Naive Bayes, and K-Nearest Neighbors (KNN)**—are applied to the dataset.

The performance of these algorithms is evaluated using **accuracy, precision, recall, and F1-score**. The results are compared using tables and graphs to determine which algorithm provides the best performance. This study helps in understanding the effectiveness of different machine learning algorithms for wine classification.

2.1 Overview of Wine Dataset

The **Wine Dataset** is a popular dataset used in machine learning for classification and data analysis. It contains chemical properties of wine samples that can be used to identify different categories of wine. The dataset consists of **178 wine samples, 13 input features, and 3 different wine classes**.

The important characteristics/features include:

- Alcohol
- Malic acid
- Ash
- Alcalinity of ash
- Magnesium
- Total phenols
- Flavanoids
- Nonflavanoid phenols
- Proanthocyanins
- Color intensity
- Hue
- OD280/OD315 of diluted wines
- Proline

In this project, these **chemical characteristics are used as input features** for machine learning models. The target variable represents the category of wine:

- **0 – Class 1**
- **1 – Class 2**
- **2 – Class 3**

2.2 Role of Machine Learning in Wine Classification

Machine Learning plays an important role in analyzing wine characteristics and classifying wine samples into different categories. By using chemical properties such as alcohol,

magnesium, flavanoids, color intensity, and proline, machine learning algorithms can identify patterns within the dataset.

In this project, Machine Learning is used to:

- **Analyze** the chemical characteristics of wine samples.
- **Classify** wine samples into their appropriate categories.
- **Compare** the performance of different classification algorithms.
- **Predict** the class of new or unseen wine samples.
- **Evaluate** model performance using accuracy, precision, recall, and F1-score.

3. System Requirements

The **Wine Dataset Classification Using Machine Learning** system requires basic hardware and software resources for data preprocessing, model training, execution, and performance evaluation. Since the Wine Dataset is relatively small, the project can be implemented efficiently on a standard laptop or desktop computer without requiring high-performance hardware.

3.1 Hardware Requirements

The hardware requirements for the proposed **Wine Dataset Classification Using Machine Learning** system are as follows:

- **Processor:** Intel Core i3 or higher.
- **RAM:** Minimum 4 GB RAM; 8 GB or more is recommended.
- **Hard Disk:** At least 10 GB of available storage space.
- **Operating System:** Windows 10/11, Linux, or macOS.

The Wine Dataset is relatively small and does not require a dedicated GPU or high-performance computing system.

3.2 Software Requirements

The software requirements for implementing the **Wine Dataset Classification Using Machine Learning** system are as follows:

- **Google Colab:** Used as the development and execution environment for writing, running, and testing the Python program.
- **Pandas:** Used for loading, organizing, analyzing, and manipulating the Wine Dataset.
- **NumPy:** Used for numerical calculations and handling arrays and data structures.
- **Matplotlib:** Used for creating graphs and visualizing the performance comparison of the algorithms.
- **Seaborn:** Can be used for statistical visualization and creating confusion matrix heatmaps.
- **Scikit-learn:** Used for data preprocessing, machine learning algorithms, train-test splitting, and evaluation metrics. It provides classifiers such as **Decision Tree, Naive Bayes, and K-Nearest Neighbors (KNN)**.
- **CSV Dataset:** The downloaded Wine Dataset CSV file is used as the input dataset for training and testing the machine learning models.

4. DATASET ANALYSIS AND PREPROCESSING

The dataset and preprocessing stages are important parts of the **Wine Dataset Classification** system. Proper preprocessing ensures that the machine learning algorithms receive clean, organized, and suitable data for training and testing. This project uses the **Wine Dataset**, which contains chemical properties of wine samples and their corresponding wine classes.

4.1 Dataset Overview

The dataset used in this project is the **Wine Dataset**, which is a structured dataset containing information about the chemical characteristics of different wine samples. Each row in the dataset represents a wine sample, while each column represents a specific chemical property or the target class.

The dataset contains **178 wine samples and 13 input**. The target variable represents **three different classes of wine**.

- **0 – Class 1**
- **1 – Class 2**
- **2 – Class 3**

4.2 Data Loading and Inspection

The Wine Dataset is loaded into the system using **Pandas** and stored in a DataFrame. The first few records of the dataset are displayed to understand the data values and structure. The column names, data types, and statistical information are also examined to understand the characteristics of the dataset and identify any possible missing or inconsistent values. This inspection helps ensure that the dataset is properly prepared for the preprocessing and machine learning stages.

4.3 Feature and Target Separation

The Wine Dataset is divided into **input features (X)** and the **target variable (y)**. The input features contain the chemical characteristics of the wine samples, while the target variable represents the class of each wine sample.

- **X = Input Features**
- **y = Target Variable**

The feature data is then used to train the **Decision Tree, Naive Bayes, and KNN** models for wine classification.

4.4 Train-Test Split

The Wine Dataset is divided into **80% training data and 20% testing data**. The training data is used to train the machine learning models, while the testing data is used to evaluate their performance on unseen wine samples. The **random_state** parameter is used to ensure that the same data split is obtained each time the program is executed. The **stratify=y** parameter maintains a similar distribution of the three wine classes in both the training and testing datasets. This ensures a fair and reliable evaluation of the models.

4.5 Feature Scaling

Feature scaling is performed to bring the numerical features of the Wine Dataset to a similar scale. This is particularly important for algorithms such as **K-Nearest Neighbors (KNN)**, where distance calculations are affected by the magnitude of the features. In this project, **StandardScaler** is used to standardize the features for KNN and Naive Bayes.

5. MACHINE LEARNING ALGORITHMS

Machine learning algorithms analyze the chemical characteristics of wine samples and learn patterns that can be used to classify them into different wine categories.

In this project, three supervised machine learning classification algorithms are used: **Decision Tree Classifier, Gaussian Naive Bayes, and K-Nearest Neighbors (KNN)**. These algorithms are trained using the wine samples in the training dataset and are then tested using unseen data.

5.1 Decision Tree Classifier

A **Decision Tree Classifier** is a supervised machine learning algorithm commonly used for classification problems. It works by dividing the dataset into smaller groups using a series of decision rules based on the input features.

Working of Decision Tree

The Decision Tree starts with a **root node**, which contains the complete dataset. It selects an appropriate feature and a threshold to split the data into different branches. The splitting process continues until the data is sufficiently separated into different classes or a stopping condition is reached. Each internal node represents a decision based on a feature, while each leaf node represents the predicted wine class.

In this project, the Decision Tree uses the chemical characteristics of wine samples to identify patterns and classify them into one of the three wine classes.

5.2 K-Nearest Neighbors (KNN)

K-Nearest Neighbors (KNN) is a supervised machine learning algorithm used for classification and regression problems. In classification, KNN predicts the class of a new data point based on the classes of its nearest neighboring data points.

Working of KNN

When a new wine sample is provided to the KNN model, the algorithm calculates the distance between the new sample and the existing training samples. It then identifies the **K nearest data points**.

In this project, the value of **K is set to 5**, meaning that the five closest training samples are considered for making the prediction. The wine class that occurs most frequently among these neighbors is selected as the predicted class.

5.3 Decision Tree Classifier

A Decision Tree Classifier is a supervised machine learning algorithm used for classification. It represents the decision-making process in the form of a tree structure. The tree consists of:

- Root node
- Decision nodes
- Branches
- Leaf nodes

The root node represents the first feature used to divide the data. Internal decision nodes represent further conditions, while leaf nodes represent the final predicted classes.

Working of Decision Tree

The Decision Tree starts with the complete Wine Dataset at the root node. It selects a suitable feature and threshold to divide the wine samples into smaller groups based on their chemical characteristics. The splitting process continues recursively until suitable stopping conditions are reached.

For example, the tree may first divide wine samples based on **alcohol content**. Each resulting group may then be divided using another feature such as **flavanoids, color intensity, or proline**. Eventually, the tree reaches a leaf node that represents the predicted wine class.

The tree therefore makes a prediction by following a sequence of decisions from the root node to a leaf node.

Gini Impurity

The Decision Tree in this project uses the **Gini criterion** to determine the quality of a split. Gini impurity measures how mixed the different wine classes are within a node.

A node containing samples from only one wine class has low impurity, while a node containing samples from multiple classes has higher impurity. The Decision Tree attempts to select splits that reduce impurity and create groups containing more similar target classes.

The Gini impurity can be represented as:

$$\text{Gini} = 1 - \sum(p_i^2)$$

where p_i represents the proportion of samples belonging to class i .

A lower Gini impurity indicates a purer node, meaning that most samples in that node belong to the same wine class.

Maximum Depth

The Decision Tree in this project uses a **maximum depth of 4**. Maximum depth controls how deep the tree can grow.

Limiting the depth prevents the tree from becoming unnecessarily complex and helps reduce the possibility of overfitting. A very deep tree may memorize the training data instead of learning general patterns.

6. PERFORMANCE EVALUATION METRICS

Performance evaluation is an important stage in the **Wine Dataset Classification Using Machine Learning** system. After training the Decision Tree, KNN, and Naive Bayes models, their performance needs to be measured to determine how accurately they classify wine samples into their respective classes.

In this project, several evaluation metrics are used to compare the performance of the three machine learning algorithms. The main metrics are **Accuracy, Precision, Recall, F1 Score, Confusion Matrix, and Classification Report**

6.1 Accuracy

Accuracy measures the overall percentage of predictions that are correctly classified by the model. It is calculated as the number of correct predictions divided by the total number of predictions.

$$\text{Accuracy} = \text{Number of Correct Predictions} / \text{Total Number of Predictions}$$

For binary classification, accuracy can be represented as:

$$\text{Accuracy} = (\text{TP} + \text{TN}) / (\text{TP} + \text{TN} + \text{FP} + \text{FN})$$

where:

- **TP (True Positive):** The model correctly predicts a sample as belonging to a particular wine class.
- **TN (True Negative):** The model correctly predicts that a sample does not belong to a particular wine class.
- **FP (False Positive):** The model incorrectly predicts a sample as belonging to a particular wine class.
- **FN (False Negative):** The model incorrectly predicts that a sample does not belong to a particular wine class when it actually belongs to that class.

6.2 Precision

Precision measures how accurately the model identifies samples predicted as belonging to a particular class.

The formula is:

$$\text{Precision} = \text{TP} / (\text{TP} + \text{FP})$$

A high precision means that fewer samples are incorrectly assigned to the predicted class.

For example, if the model predicts several wine samples as **Class 1**, precision indicates how many of those predicted Class 1 samples actually belong to Class 1.

6.3 Recall

Recall measures how effectively the model identifies the actual samples belonging to a particular class.

The formula is:

$$\text{Recall} = \text{TP} / (\text{TP} + \text{FN})$$

A high recall means that the model correctly identifies a larger proportion of the actual samples belonging to that class.

For example, if several wine samples actually belong to **Class 2**, recall indicates how many of those samples are correctly identified as Class 2 by the model.

6.4 F1 Score

The F1 Score combines precision and recall into a single performance measure. It is calculated using the harmonic mean of precision and recall.

$$\text{F1 Score} = 2 \times (\text{Precision} \times \text{Recall}) / (\text{Precision} + \text{Recall})$$

A higher F1 Score indicates that the model provides a good balance between precision and recall.

6.5 Confusion Matrix

A confusion matrix is a table-like representation used to evaluate the predictions of a classification model.

Since the Wine Dataset contains **three classes**, the confusion matrix is a **3 × 3 matrix**. It compares the actual wine classes with the classes predicted by the model.

The rows represent the **actual classes**, while the columns represent the **predicted classes**.

For example, if an actual Class 1 wine sample is predicted as Class 2, it represents a misclassification between those two classes.

6.6 Classification Report

The classification report provides a detailed summary of the classification performance of a machine learning model.

It generally includes:

- **Precision**
- **Recall**
- **F1 Score**
- **Support**

The classification report provides these values for each target class.

In this project, the report evaluates:

- **Class 0**
- **Class 1**
- **Class 2**

The **support** value represents the number of actual samples belonging to each class in the test dataset.

6.7 Importance of Evaluation Metrics in Wine Classification

Evaluation metrics are important in wine classification because accuracy alone may not provide a complete picture of model performance. A model may achieve high overall accuracy while performing better on some wine classes than others.

Therefore, **precision, recall, F1 Score, and confusion matrix** results are also considered when comparing the three algorithms.

Using multiple evaluation metrics helps identify the strengths and weaknesses of each model and provides a fair comparison between **Decision Tree, KNN, and Naive Bayes**.

The model with the best overall evaluation results is considered the **best-performing algorithm** for the Wine Dataset classification project.

7. IMPLEMENTATION

The **Wine Dataset Classification system** is implemented using Python and the **Scikit-learn** library. The implementation involves loading and preprocessing the Wine Dataset, training three machine learning classification models, generating predictions, evaluating their performance, and visualizing the results.

7.1 Environment Setup

The system is implemented using **Python** and can be executed using **Google Colab, Jupyter Notebook, or Visual Studio Code**. The major libraries used are **NumPy, Pandas, Matplotlib, Seaborn, and Scikit-learn**.

Google Colab provides a convenient environment for executing the Python code without requiring extensive software installation. The Wine Dataset is loaded into a Pandas DataFrame and prepared for model training and testing.

7.2 Model Implementation

Three machine learning classification algorithms are implemented in the proposed system:

- **Decision Tree Classifier** is implemented using the **Gini criterion** with a maximum depth of **4**.
- **K-Nearest Neighbors (KNN)** classifier is implemented with **K = 5** using standardized features.
- **Gaussian Naive Bayes** classifier is implemented using the probability-based classification approach.

The models are trained using the training dataset and are then used to generate predictions for the testing dataset. The same training and testing datasets are used for all three algorithms to provide a fair comparison.

7.3 Model Evaluation

The performance of the implemented models is evaluated using **Accuracy, Precision, Recall, F1 Score, Confusion Matrix, and Classification Report**.

These metrics are used to compare the classification performance of **Decision Tree, KNN, and Naive Bayes**. Since the Wine Dataset contains three target classes, the evaluation metrics help determine how effectively each model classifies samples belonging to each wine class.

The model with the best overall performance across the evaluation metrics is considered the most suitable algorithm for the Wine Dataset.

7.4 Visualization

The confusion matrices of the three models are visualized using **Seaborn heatmaps**. These visualizations help identify correctly and incorrectly classified wine samples for each class.

The Decision Tree is also visualized as a **tree diagram** to show its decision-making structure and the sequence of feature-based decisions used for classification.

Performance comparison graphs can also be created to visually compare the **Accuracy, Precision, Recall, and F1 Score** of the three algorithms.

7.5 Implementation Code

The Python implementation used for **data loading, preprocessing, model training, prediction, evaluation, and visualization** is provided below.

The results obtained from the three machine learning algorithms are compared using suitable evaluation metrics. This comparison is used to identify the **best-performing algorithm for the given Wine Dataset**.

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

from sklearn.tree import DecisionTreeClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.neighbors import KNeighborsClassifier

from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    classification_report
)

```

```

file_path = "/content/Wine dataset.csv"
df = pd.read_csv(file_path)

```

```

print("\nFirst 5 rows of the dataset:")
display(df.head())

print("\nShape of the dataset:")
print(df.shape)

print("\nColumn names:")
print(df.columns.tolist())

```

```

print("\nDataset Information:")
df.info()

print("\nMissing Values:")
print(df.isnull().sum())

```

```

df.columns = df.columns.str.strip()

print("\nCleaned Column Names:")
print(df.columns.tolist())

```

```

X = df.drop("class", axis=1)
y = df["class"]
print("\nFeatures:")
print(X.head())

print("\nTarget:")
print(y.head())

print("\nNumber of features:", X.shape[1])
print("\nNumber of samples:", X.shape[0])

```

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("\nTraining data shape:", X_train.shape)
print("Testing data shape:", X_test.shape)
```

```
scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)

X_test_scaled = scaler.transform(X_test)
```

```
decision_tree = DecisionTreeClassifier(
    random_state=42
)

naive_bayes = GaussianNB()

knn = KNeighborsClassifier(
    n_neighbors=5
)
```

```
decision_tree.fit(
    X_train_scaled,
    y_train
)

naive_bayes.fit(
    X_train_scaled,
    y_train
)

knn.fit(
    X_train_scaled,
    y_train
)
```

```
y_pred_dt = decision_tree.predict(X_test_scaled)

y_pred_nb = naive_bayes.predict(X_test_scaled)

y_pred_knn = knn.predict(X_test_scaled)
```

```
dt_accuracy = accuracy_score(y_test, y_pred_dt)

dt_precision = precision_score(
    y_test,
    y_pred_dt,
    average="weighted"
)

dt_recall = recall_score(
    y_test,
    y_pred_dt,
    average="weighted"
)

dt_f1 = f1_score(
    y_test,
    y_pred_dt,
    average="weighted"
)
```

```

nb_accuracy = accuracy_score(y_test, y_pred_nb)

nb_precision = precision_score(
    y_test,
    y_pred_nb,
    average="weighted"
)

nb_recall = recall_score(
    y_test,
    y_pred_nb,
    average="weighted"
)

nb_f1 = f1_score(
    y_test,
    y_pred_nb,
    average="weighted"
)

```

```

knn_accuracy = accuracy_score(y_test, y_pred_knn)

knn_precision = precision_score(
    y_test,
    y_pred_knn,
    average="weighted"
)

knn_recall = recall_score(
    y_test,
    y_pred_knn,
    average="weighted"
)

knn_f1 = f1_score(
    y_test,
    y_pred_knn,
    average="weighted"
)

```

```

results = pd.DataFrame({
    "Algorithm": [
        "Decision Tree",
        "Naive Bayes",
        "KNN"
    ],
    "Accuracy": [
        dt_accuracy,
        nb_accuracy,
        knn_accuracy
    ],
    "Precision": [
        dt_precision,
        nb_precision,
        knn_precision
    ],
    "Recall": [
        dt_recall,
        nb_recall,
        knn_recall
    ],
})

```

```

        "F1 Score": [
            dt_f1,
            nb_f1,
            knn_f1
        ]
    })

print("\n=====PERFORMANCE COMPARISON=====")
display(results)

```

```

results_percentage = results.copy()

results_percentage[
    ["Accuracy", "Precision", "Recall", "F1 Score"]
] = (
    results_percentage[
        ["Accuracy", "Precision", "Recall", "F1 Score"]
    ] * 100
).round(2)

print("\n=====PERFORMANCE COMPARISON IN PERCENTAGE=====")
display(results_percentage)

```

```

▶ print("\n=====CLASSIFICATION REPORT - DECISION TREE=====")

print(
    classification_report(
        y_test,
        y_pred_dt
    )
)

print("\n=====CLASSIFICATION REPORT - NAIVE BAYES=====")

print(
    classification_report(
        y_test,
        y_pred_nb
    )
)

print("\n=====CLASSIFICATION REPORT - KNN=====")
print(
    classification_report(
        y_test,
        y_pred_knn
    )
)

```

```

▶ cm_dt = confusion_matrix(
    y_test,
    y_pred_dt
)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm_dt,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.title("Confusion Matrix - Decision Tree")

plt.xlabel("Predicted Class")

plt.ylabel("Actual Class")

plt.show()

```

```

cm_nb = confusion_matrix(
    y_test,
    y_pred_nb
)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm_nb,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.title("Confusion Matrix - Naive Bayes")

plt.xlabel("Predicted Class")

plt.ylabel("Actual Class")

plt.show()

```

```

cm_knn = confusion_matrix(
    y_test,
    y_pred_knn
)

plt.figure(figsize=(6, 5))

sns.heatmap(
    cm_knn,
    annot=True,
    fmt="d",
    cmap="Blues"
)

plt.title("Confusion Matrix - KNN")

plt.xlabel("Predicted Class")

plt.ylabel("Actual Class")

plt.show()

```

```

▶ results_percentage.set_index(
    "Algorithm"
)[
    ["Accuracy", "Precision", "Recall", "F1 Score"]
].plot(
    kind="bar",
    figsize=(10, 6)
)

plt.title(
    "Performance Comparison of ML Algorithms"
)

plt.xlabel("Algorithms")

plt.ylabel("Performance (%)")

plt.xticks(rotation=0)

plt.ylim(0, 100)

plt.legend(
    title="Evaluation Metrics"
)

plt.grid(
    axis="y"
)

plt.show()

```

Activate Windows
Go to Settings to activate Windows.

```

best_index = results["F1 Score"].idxmax()

best_algorithm = results.loc[
    best_index,
    "Algorithm"
]

best_accuracy = results.loc[
    best_index,
    "Accuracy"
]

best_precision = results.loc[
    best_index,
    "Precision"
]

best_recall = results.loc[
    best_index,
    "Recall"
]

best_f1 = results.loc[
    best_index,
    "F1 Score"
]

```

Activate Windows

```

print("\n")
print("=====FINAL RESULTS=====")

print(
    "\nBest Performing Algorithm:",
    best_algorithm
)

print(
    "Accuracy:",
    round(best_accuracy * 100, 2),
    "%"
)

print(
    "Precision:",
    round(best_precision * 100, 2),
    "%"
)

print(
    "Recall:",
    round(best_recall * 100, 2),
    "%"
)

print(
    "F1 Score:",
    round(best_f1 * 100, 2),
    "%"
)

print("\n=====CONCLUSION=====")

print(
    f"\nAmong Decision Tree, Naive Bayes and KNN, "
    f"{best_algorithm} performed the best based on "
    f"the highest F1 Score."
)

```

8. RESULTS AND DISCUSSION

The Results and Discussion chapter presents the performance of the machine learning models used for **Wine Dataset classification**. Three classification algorithms were used in this project: **Gaussian Naive Bayes, K-Nearest Neighbors (KNN), and Decision Tree Classifier**.

8.1 Dataset Preview

The first five records of the **Wine Dataset** were displayed using the `data.head()` function.

First 5 rows of the dataset:

	class	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols	Proanthocyanins	Color intensity	Hue	OD280/OD315 of diluted wines
0	1	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92
1	1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40
2	1	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17
3	1	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45
4	1	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93

Figure 8.1: Dataset Preview – First Five Rows

The displayed output shows the first five records along with the **chemical features of the wine samples** used for classification. This provides a basic understanding of the structure and values of the Wine Dataset before applying preprocessing techniques and machine learning algorithms.

8.2 Dataset Shape and Testing Data

The shape of the **Wine Dataset** was checked using the `data.shape` function.

Shape of the dataset:
(178, 14)

Figure 8.2: Dataset Shape

The dataset shape represents the total number of rows and columns. The number of rows represents the total number of records, while the number of columns represents the available attributes.

8.3 Model Performance Results

The three machine learning algorithms, **Gaussian Naive Bayes**, **K-Nearest Neighbors (KNN)**, and **Decision Tree**, were trained using the Wine Dataset and tested using the same testing data.

The performance of all three models was evaluated using **Accuracy**, **Precision**, **Recall**, and **F1 Score**. The calculated results were combined into a single performance comparison table, making it easier to compare the algorithms based on the selected evaluation metrics.

The comparison helps identify which algorithm provides better classification performance for the Wine Dataset. The model with the highest overall performance across the evaluation metrics is considered the **best-performing algorithm** for this project.

=====PERFORMANCE COMPARISON=====

	Algorithm	Accuracy	Precision	Recall	F1 Score
0	Decision Tree	0.944444	0.951389	0.944444	0.944961
1	Naive Bayes	0.972222	0.974359	0.972222	0.972263
2	KNN	0.972222	0.974747	0.972222	0.972369

=====PERFORMANCE COMPARISON IN PERCENTAGE=====

	Algorithm	Accuracy	Precision	Recall	F1 Score
0	Decision Tree	94.44	95.14	94.44	94.50
1	Naive Bayes	97.22	97.44	97.22	97.23
2	KNN	97.22	97.47	97.22	97.24

8.4 Classification Report

Decision Tree

=====CLASSIFICATION REPORT - DECISION TREE=====

	precision	recall	f1-score	support
1	1.00	0.92	0.96	12
2	0.88	1.00	0.93	14
3	1.00	0.90	0.95	10
accuracy			0.94	36
macro avg	0.96	0.94	0.95	36
weighted avg	0.95	0.94	0.94	36

Naïve Bayes

```
=====CLASSIFICATION REPORT - NAIVE BAYES=====
```

	precision	recall	f1-score	support
1	0.92	1.00	0.96	12
2	1.00	0.93	0.96	14
3	1.00	1.00	1.00	10
accuracy			0.97	36
macro avg	0.97	0.98	0.97	36
weighted avg	0.97	0.97	0.97	36

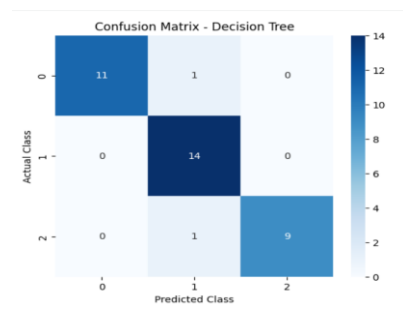
KNN

```
=====CLASSIFICATION REPORT - KNN=====
```

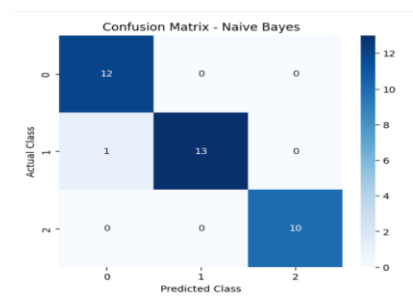
	precision	recall	f1-score	support
1	1.00	1.00	1.00	12
2	1.00	0.93	0.96	14
3	0.91	1.00	0.95	10
accuracy			0.97	36
macro avg	0.97	0.98	0.97	36
weighted avg	0.97	0.97	0.97	36

8.5 Confusion Matrix

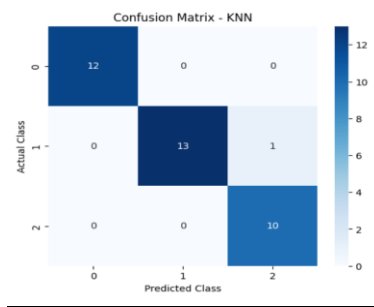
Decision Tree



Naïve Bayes



KNN



8.6 Discussion of Results

The results obtained from the models were presented together in a **model performance comparison table**. This makes it easier to compare the classification performance of the three algorithms across the different evaluation metrics.

The **Accuracy** indicates the overall percentage of wine samples correctly classified by each model. **Precision** shows how accurately the models classify samples into the predicted wine classes, while **Recall** measures how effectively the models identify the actual samples belonging to each class. The **F1 Score** provides a balance between precision and recall.

The confusion matrices and classification reports provide additional information about the correctly and incorrectly classified wine samples. These results help in understanding the strengths and weaknesses of each algorithm.

Based on the comparison of the evaluation metrics, the algorithm that achieves the **best overall performance** is selected as the best-performing model for the Wine Dataset classification task. Therefore, the final model selection is based on the actual results obtained from **Naive Bayes, KNN, and Decision Tree**, rather than considering only a single evaluation metric.

9. CONCLUSION

The **Wine Dataset Classification Using Machine Learning** project successfully demonstrated the application of machine learning classification algorithms to classify wine samples based on their chemical characteristics. Three algorithms, **Gaussian Naive Bayes, K-Nearest Neighbors (KNN), and Decision Tree Classifier**, were implemented and evaluated using the same training and testing data.

The performance of the models was compared using **Accuracy, Precision, Recall, F1 Score, Confusion Matrix, and Classification Report**. The results provided a detailed understanding of how each algorithm performs in classifying the three different wine classes.

The project also demonstrated the importance of data preprocessing, feature scaling, model training, and performance evaluation in developing a reliable machine learning classification system. The comparison of the algorithms helps identify the model that provides the best overall performance for the given Wine Dataset.

Based on the comparison of the obtained results, the algorithm with the best overall performance can be selected as the most suitable model for the Wine Dataset. The project concludes that machine learning provides an effective approach for **classifying wine samples into different categories using their chemical characteristics**.

The results of the three algorithms were presented together in a **performance comparison table and graphical representation**. This made it easier to compare the models using the same evaluation criteria. The comparison helps in identifying the strengths and limitations of each algorithm and selecting the most suitable model for the given dataset.

Overall, the project shows that machine learning algorithms can effectively be used for **wine classification based on chemical properties**. The best-performing model can be selected from the three algorithms based on the actual evaluation results obtained from the implementation.